



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The state space is the set of all possible states the agent can be in — in my grid, every non-wall cell (x, y). Each state exists only once, and states are connected like a graph. The search tree is what the algorithm builds while exploring, starting from the start state. The same state can appear many times in the tree because there are many paths to the same cell (e.g. (5,3) can be reached by Right→Up or Up→Right). So the state space is small and fixed, but the search tree can be much bigger, even infinite.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

It solves the problem of repeated states. In a grid, every move can be undone, so without memory, the search would keep revisiting the same cells ((4,2) → (5,2) → (4,2) ...). The reached set makes sure each cell is expanded only once, turning a tree search into a graph search. Without it, my DFS agent would dive into a cycle and never terminate, because DFS always goes deeper and would keep pushing the cell it just came from. With it, DFS ended with a 44-step path instead of looping forever.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

In my runs, BFS found the shortest route (8 moves from (4,2) to (2,8)) and collected all 5 pellets (score 100). DFS took 44 moves for the same goal and, in the GUI, used 27 moves to reach a pellet only 3 cells away, finishing with a score of 20.

BFS is optimal here because every move costs the same (1). It explores the grid in layers — distance 1, then 2, then 3 — so the first time it reaches the goal, that must be the fewest-move path. DFS ignores depth completely; it follows one branch as deep as possible and only backs up when stuck, so the first path it finds is whatever it stumbles on, which is usually long and winding.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

From the lecture: BFS has space $O(b^d)$ and UCS $O(b^{(1+\lceil C^*/\epsilon \rceil)})$ (same as BFS when all costs are 1), while DFS has space only $O(b \cdot m)$.

BFS/UCS must keep the whole frontier and the reached set in memory at once. On a 1000×1000 grid with food far away, that can be hundreds of thousands of cells, which is what would make them crash. DFS only stores the current path and its untried siblings, so its memory stays small even on a huge grid. The trade-off: BFS/UCS guarantee the shortest path but use lots of memory; DFS uses little memory but gives no guarantee about path length.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING